String & String Builder.

Q. What is string?

- String is a collection of character. ~~for~~
  Eg: "Abhi".
- String will be in double quote.
- It's a non-primitive datatype.

Syntax:-

String name = "Abhi Aditya".

- everything with capital letter is a class.

- It is the most commonly used class in JAVA's class library.
- Every string created is an object.

String name = "Kunal"

- String → Datatype
- name → Reference Variable
- "Kunal" → Object.

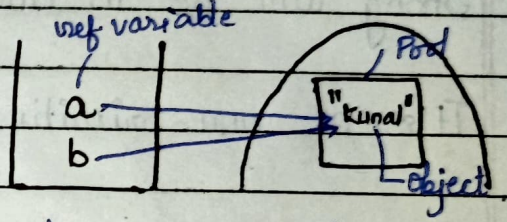
Date
Page

★ Concept :-

- **String Pool :-** It is a seperate memory structure inside the heap memory.

- **Why Sperate pool ?**
  All the similar values of string are not ~~like~~ recreated in the pools.

Eg :- string a = "Kunal"
string b = "Kunal"



- a will create "kunal" object
- b will check in pool as it already exist it will point its pre existing "kunal" Pool

★ use case :- It makes our programs more optimized

Note :-

* If we try to change this object via ~~this~~ 'b' reference variable it will not be changed for a as strings are ~~imm~~ immutable ~~bcs~~.

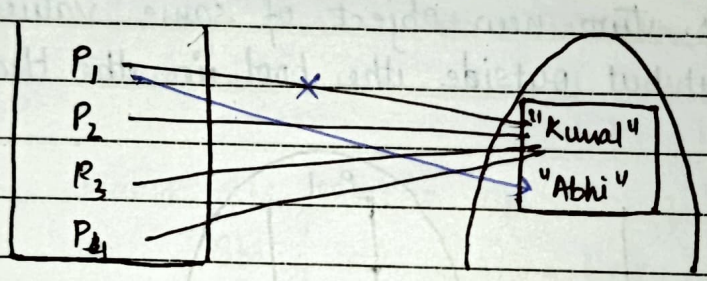
Date
Page

- Immutability.

- Strings are immutable in java, we can't change the object or modify objects.

- If you wanna rename "kunal" to "Abhi" you have to create a new object for that.

- Strings are immutable for security purpose.

- 

- 1st all person are named. kunal.
- If P_1 changed his name from "kunal" to "Abhi"
- If string would not immutibility then all the person name would have changed to "Abhi".
- But here an object "Abhi" is created & P_1 remove it pointing from "kunal" & shift it to "Abhi".

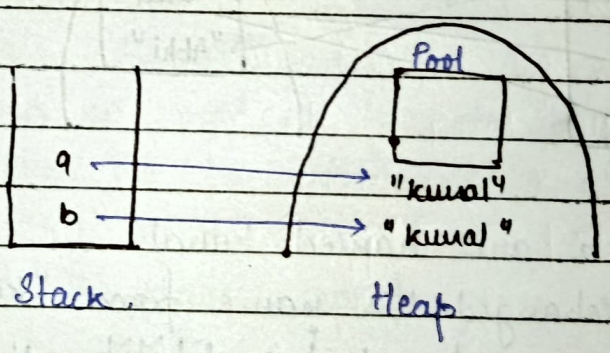
★ How to create diff object of same value.

- By using new keyword we can create a new object.

- String a = new string ("kunal")

  String b = new string ("kunal")

- this creates two new object of same value "kunal" but outside the pool in the Heap memory



★ Comparision Of Strings :-

- == method. :-

a ⟶ "kunal"
b ⟶ "kunal"
} a==b → false

a ⟶ "kunal"
b ⟶ "kunal"
} a==b → true.

- '==' method compare both value & ~~wher~~ whether the reference variable is pointing to same ~~ob~~ object.

- it gives true when values are equal & reference variable points to same object.

- Checking only values :-

- To check ~~only~~ that both reference variable have same value we use .equal method.

- Eg:- ~~name~~ String n1 = new String("Abhi")
  String n2 = new String("Abhi")

  n1.equal(n2) → true.

- It don't care about whether they (reference variable) are pointing to same objects or not?

★ How to access character at different index in String.

- We can't access directly like in array.
  n1[0] = x

- There is a method .charAt(n);
  n = index.
  n1.charAt(0) = A

★ System.out.println("Abhi");

I variable of itype printstream

↓

It is a class it has method in it called println.

★ How it work internally.

- println ⇒ method of printstream class that accepts various data types.
- Step 1 :- String.valueOf(obj) → coverts obj to string
- Step 2 :- If obj = null print "null".

  else call obj.toString() → prints the string

★ Function Overloading :-

It means defining multiple methods with same name but different parameters.

- Number of Parameter
- Type of parameter
- Order of parameter.

Eg:-
```
int add(int a, int b){
    return a+b; }
int add(int a, int b ; int c){
    return a+b+c; }
```

- at compile time, it decide which function to run.

BARHATI

Date
Page

★ Pretty Printing :-

Pretty printing means formatting your output in clean, readable & well-aligned way - especially useful for printing tables, numbers, JSON, etc

Eg:- float a = 453.12345f;
system.out.printf("Formatted number is %.2f", a);

>> 453.12

% means place holder
.2f ⇒ how many decimal value do it give.
- It also rounds off.

- for PI :-

Math.PI

Eg:-
System.out.printf("Hello, I am %s & I am a %s", "Abhi", "Student

>> Hello; I am Abhi & I am a student.

Order of placeholders & variable must be same.

Date
Page

★ Common format specifier :-

%c :- character
%d :- decimal (base 10)
%e :- Exponential floating point
%f :- floating-point number
%i :- integer (base 10)
%o :- octal number (base 8)
%s :- string
%u :- unsigned decimal (integer) number
%x :- Hexa decimal (base 16)
%t :- Date/time
%n :- newline (platform independent)

\n :- newline (dependent)
works on UNIX ; Mac ; Linux.

★ Operator :-

- When we add two character. char. get converted to unicode or Ascii value then added.

Eg. sout('a' + 'b'); >> 195

- when we add anything to string everything is a string & added.

Eg :- sout("a" + 'a'); >> aa

★ Operator Overloading :— In Java, operator overloading is not supported. We can not define new behaviour for operator.

Why not supporting?

- Simplicity & readability
- Maintainability
- Avoid hidden logic.

- When we add two string we get a new object of added string is created.

★ StringBuilder Class

- StringBuilder Class is used to create & manipulate mutable string efficiently.

Why String Builder?

- Strings are immutable.
- Repeated string concatenation creates multiple string objects in memory (slow & memory heavy)
- StringBuilder solve this modifying string in place.

Syntax :—

```
StringBuilder sb = new StringBuilder();   // enter initial string if empty leave emtty

sb.append( );
   └ add string
```

Methods that String provide :

1. .toCharArray() :- It converts string into char array.

   Eg. String name = "Abhi"
   sout (Arrays.toString (name.toCharArray()));
   >> ['A', b, h, i]

   - Space will also be an element of array

2. .length() :- Gives length of string

3. .toLowerCase() :- Convert in lower case.

4. .indexOf() :- Give index value from start

5. .lastIndexOf :- Give index value from last.

6. .strip() :- White spaces are removed.

7. .split() :- splits over regex
   regex

★ Palindrome :-

Algorithm :-

- If (string == null or length == 0)
  return true.

- for (int i=0; i < str.length/2 ; i++)
  start = i end = str.length-1-i
  if start ≠ end
  return false

return true.

a b c d
s e

s = e
s++; e--

s ≠ e
false